# Codec for JsonSchema

## General Concepts

We have implemented a custom `CodecDeserializer<EPackage>` and a custom `CodecDeserializer<EPackage>` which are responsible, respectively, for the deserialization and serialization of an `EPackage` from and to a `jsonschema`.

These custom classes can be passed to the codec via the loading and saving options:

```
private Map<String, Object> getLoadOptions() {
        Map<String, Object> options = new HashMap<>();
        options.put(CodecResourceOptions.CODEC_ROOT_OBJECT,
EcorePackage.eINSTANCE.getEPackage());
        Map<EClass, Map<String, Object>> classOptions = new HashMap<>();
        Map<String, Object> schemaDefOptions = new HashMap<>();

schemaDefOptions.put(CodecModelInfoOptions.CODEC_CUSTOM_DESERIALIZERS_MAP,
Map.of(EcorePackage.Literals.EPACKAGE, new JsonSchemaToEPackageDeserializer()));
        classOptions.put(EcorePackage.Literals.EPACKAGE, schemaDefOptions);
        options.put(CodecResourceOptions.CODEC_OPTIONS, classOptions);
        return options;
    }
```

```
private Map<String, Object> getSaveOptions() {
        Map<String, Object> options = new HashMap<>();
        options.put(CodecResourceOptions.CODEC_ROOT_OBJECT,
EcorePackage.eINSTANCE.getEPackage());
        options.put(ObjectMapperOptions.OBJ_MAPPER_SERIALIZATION_FEATURES_WITH,
List.of(SerializationFeature.INDENT_OUTPUT));
        Map<EClass, Map<String, Object>> classOptions = new HashMap<>();
        Map<String, Object> schemaDefOptions = new HashMap<>();
        schemaDefOptions.put(CodecModelInfoOptions.CODEC_CUSTOM_SERIALIZERS_MAP,
Map.of(EcorePackage.Literals.EPACKAGE, new EPackageToJsonSchemaSerializer()));
        classOptions.put(EcorePackage.Literals.EPACKAGE, schemaDefOptions);
        options.put(CodecResourceOptions.CODEC_OPTIONS, classOptions);
        return options;
    }
```

These are then used in the `CodecEObjectDeserializer` and `CodecEObjectSerializer`, instead of the standard deserialization/serialization mechanism.

## What is Supported

### Deserialization

We can currently deserialize a `jsonschema` into an `EPackage` with the following characteristics:

- Top level classes -> translated to `EClass`
- Top level enum -> translated to `EEnum`

- Top level array -> translated to artificial `EClass` containing an attribute called `items` of the type of the array items
- Top level data type -> translated to `EDataType`
- Single and many attributes -> translated to `EAttribute`
- Single and many references -> translated to `EReference`
- `anyOf` and `oneOf` -> translated to artificial parent class from which all the members of `anyOf` or `oneOf` inherit; annotations are added to keep track of all information
- `allOf` -> translated into inheritance
- `const` keyword -> used to determine the type of an attribute, when `type` is not provided; kept as `EAnnotation` to allow back serialization;
- `format` keyword -> kept as `EAnnotation` to allow back serialization
- `description` keyword -> added as `EAnnotation` documentation
- `$schema` keyword -> kept as `EAnnotation` to allow back serialization

We currently **DO NOT** support:

- `anyOf` / `oneOf` in case the elements are not specified by `$ref` but through complex objects

```
{
    "$schema": "http://json-schema.org/draft-07/schema#",
    "definitions": {
       "ReadResourceResult": {
            "description": "The server's response to a resources/read request
from the client.",
            "properties": {
                "contents": {
                        "anyOf": [
                            {
                                "type": "object",
                                "properties" : {
                                    "test": {
                                        "type": "boolean"
                                    },
                                    "name":  {
                                        "type": "string"
                                    }
                                }
                            },
                            {
                                "type": "object",
                                "properties" : {
                                    "test": {
                                        "type": "boolean"
                                    },
                                    "age":  {
                                        "type": "integer"
                                    }
                                }
                            }
                        ],
                    "type": "object"
```

```
                }
            },
            "required": [
                "contents"
            ],
            "type": "object"
        }
    }
}
```

- `anyOf` / `oneOf` in case the elements contain a mix of `$ref` schema and object defintions

```
{
    "$schema": "http://json-schema.org/draft-07/schema#",
    "definitions": {
       "ReadResourceResult": {
            "description": "The server's response to a resources/read request
from the client.",
            "properties": {
                "contents": {
                        "anyOf": [
                            {
                                "type": "object",
                                "properties" : {
                                    "test": {
                                        "type": "boolean"
                                    },
                                    "name":  {
                                        "type": "string"
                                    }
                                }
                            },
                            {
                                "$ref": "#/defintions/SomeClass",

                            }
                        ],
                    "type": "object"
                }
            },
            "required": [
                "contents"
            ],
            "type": "object"
        }
    }
}
```

- `const` keyword in top level array with no `items` specified
- `allOf` in which `allOf` is not an array node, or in case multiple explicit objects are defined. For instance:

  - [x]

```
{
```

```
  "definitions": {
    "Employee": {
      "allOf": [
        { "$ref": "#/definitions/Person" },
        {
          "type": "object",
          "properties": {
            "employeeId": { "type": "string" }
          },
          "required": ["employeeId"]
        }
      ]
    }
  }
}
```

- [ ]

```
{
  "definitions": {
    "Employee": {
      "allOf": [
        { "type": "object"
          "properties": {
            "employeeAge": { "type": "integer" }
          }
        },
        {
          "type": "object",
          "properties": {
            "employeeId": { "type": "string" }
          },
          "required": ["employeeId"]
        }
      ]
    }
  }
}
```

## Serializa

To serialize an `EPackage` to a `jsonschema` we make use of the `EAnnotation` defined during deserialization. If, these `EAnnotation` are not present because, for instance, no deserialization has been done and we just have an `EPackage` to be serialized into a `jsonschema` there might be some discrepancy with what comes out. The following should still work:

- Gen model `EAnnotation` as documentation is translated into a `jsonschema` `description` annotation;
- If an `EReference` type has some sub classes, then the `EReference` is translated into an `anyOf` with all the sub types possible;

## Other Important Things to Notice

- We built our own serializer/deserializer. This means that we should take care of setting the `StreamWriteContext` and `ReadWriteContext` at the right feature, if we want to keep track of the emf information. This is currently **NOT DONE!**